

Architettura degli Elaboratori

Funzioni, Ricorsione



SAPIENZA
UNIVERSITÀ DI ROMA

Alessandro Checco

checco@di.uniroma1.it

Special thanks and credits:

Andrea Sterbini, Iacopo Masi,
Claudio di Ciccio

[S&PdC]

Capitolo 2



Informazioni sull'esonero

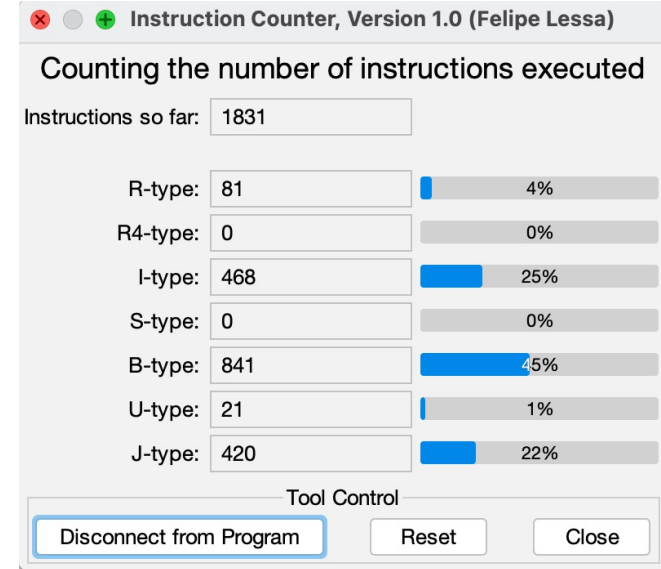
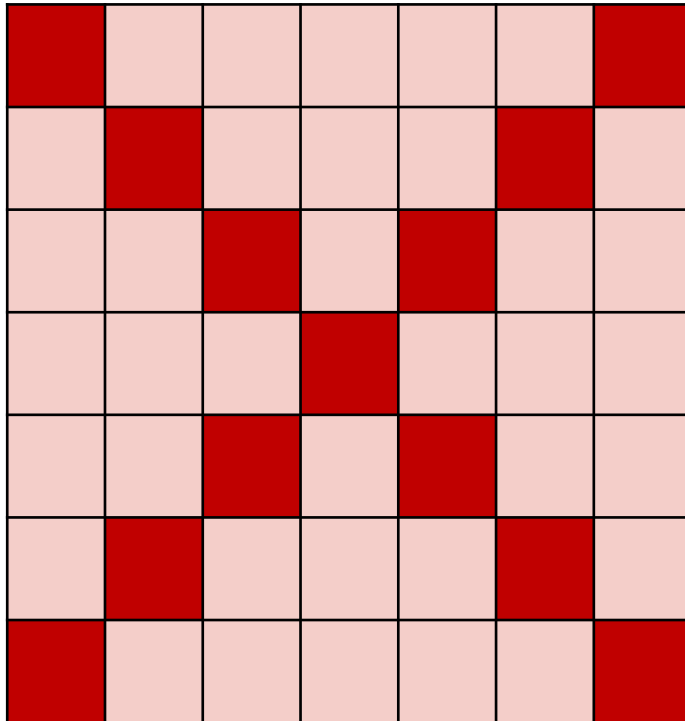
- Quando: 9 Maggio 2025 dalle 16 alle 19
- Dove: Aula 16 e 17 laboratori di via Tiburtina
- Prova pratica di assembly in rars
- Note:
 - necessaria la registrazione
 - aperto solo a chi sta frequentando le lezioni

Esercizio



SAPIENZA
UNIVERSITÀ DI ROMA

Somma della diagonale (v. più efficiente)

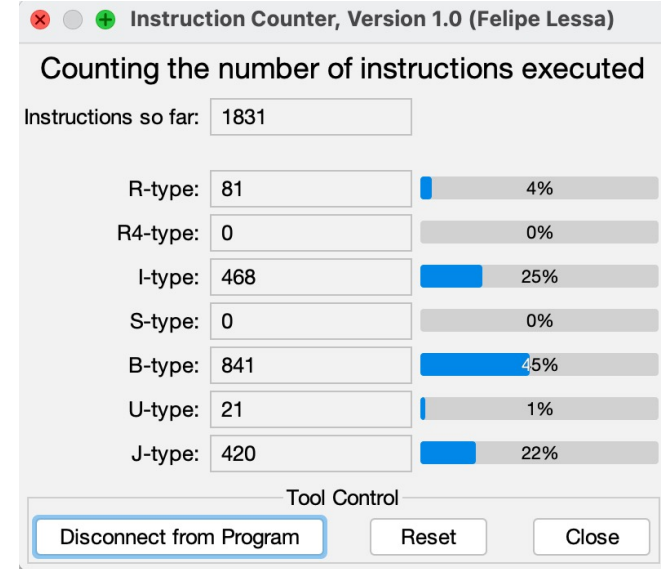
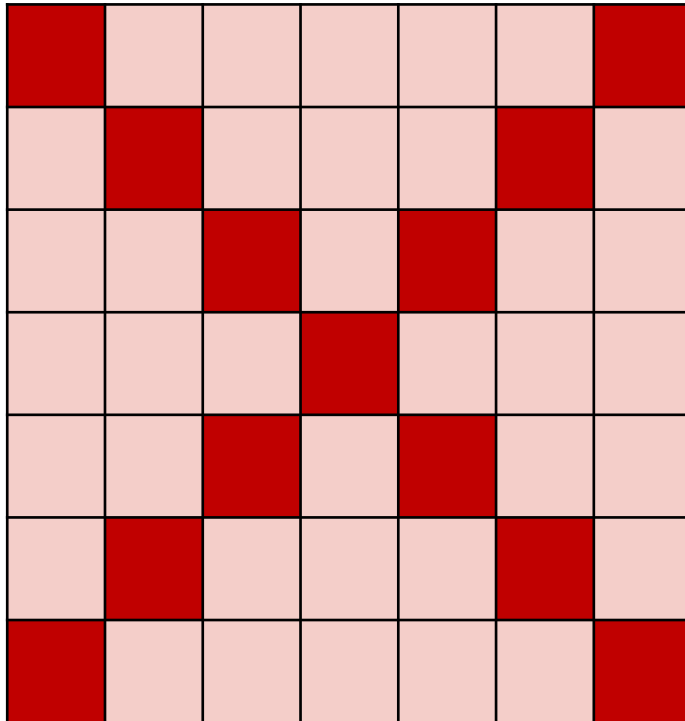


Esercizio 1:

Scrivere un programma in assembly che, data una matrice bidimensionale quadrata MATRIX di word in memoria, di lato N, restituisca in s0 il risultato della somma dei valori sulle diagonali principale e secondaria.



Somma della diagonale (v. più efficiente) / reprise



Esercizio 2:

Aggiungere istruzioni di stampa e di uscita al programma di cui all'esercizio 1.



Soluzione es. somma diagonali con funzioni

Definiamo la funzione `is_diagonal` che torna **1** (vero) se la casella a coordinate `x, y` si trova su una delle due diagonali principali altrimenti torna **0** (falso), con argomenti:

```
#    a0  coordinata x (riga)
#    a1  coordinata y (colonna)
#    a2  lato della matrice
```

is_diagonal:

```
beq    a0, a1, yes    # se x=y siamo sulla prima diagonale
add    t0, a0, a1      # altrimenti se la somma x+y+1
addi    t0, t0, 1
beq    t0, a2, yes    # è uguale al lato siamo sulla seconda
# altrimenti non siamo sulle due diagonali principali
li    a0, 0            # il risultato è 0 (falso)
jr    ra              # ritorno all'istruzione successiva
                        # alla chiamata
```

yes:

```
li    a0, 1            # il risultato è 1 (vero)
jr    ra              # ritorno all'istruzione successiva
                        # alla chiamata
```

lettura dell'elemento corrente

Definiamo una seconda funzione che legge dalla matrice l'elemento a coordinate x,y (con gli stessi argomenti dell'altra)

```
#    a0  coordinata x (riga)
#    a1  coordinata y (colonna)
#    a2  lato della matrice
```

leggi_elemento:

```
mul  t0, a0, a2          # x*LATO
add  t0, t0, a1          # y + x*LATO
slli t0, t0, 2           # offset = 4 * (y + x * LATO)
la   t1, matrice
add  t0, t0, t1
lw   a0, (t0)            # leggo matrice[x][y]
jr   ra                 # torno  l'esecuzione al chiamante
```

NOTA: cerco di non usare registri ulteriori in modo da lasciare più libertà nel programma chiamante

NOTA: stack vuoto, perché?

Programma principale

.data

```
matrice: .word    0:400          # matrice 20x20
LATO:    .word    20             # lato della matrice
```

.text

```
main:      li      a0, 0          # x = 0
           li      a1, 0          # y = 0
           lw      a2, LATO       # lato della matrice
           li      s0, 0          # somma = 0
cicloX:    beq     a0, a2, fine    # se ultima riga
cicloY:    beq     a1, a2, nextX   # se ultima colonna
           addi    sp, sp, -8
           sw      a0, 0(sp)
           sw      a1, 4(sp)
           jal     is_diagonal    # test se sulla diagonale
           mv      t0, a0
           lw      a1, 4(sp)
           lw      a0, 0(sp)
           beqz    t0, nextY       # se falso prossima colonna
```

(segue)

```
        jal    leggi_elemento    # altrimenti leggi
        add    s0, s0, a0        # e somma l'elemento
        lw     a1, 4(sp)
        lw     a0, 0(sp)
        addi   sp, sp, 8        # libero lo stack

nextY:   addi   a1, a1, 1        # y += 1 (prossima colonna)
        j      cicloY

nextX:   addi   a0, a0, 1        # x += 1 (prossima riga)
        li     a1, 0            # y = 0 (colonna 0)
        j      cicloX

fine:    mv     a0, s0          # stampo la somma
        li     a7, 1            # syscall 1 = print integer
        ecall

        li     a7, 10          # syscall 10 = fine
        ecall
```



LA RICORSIONE



LA RICORSIONE

LA RICORSIONE

LA RICORSIONE

LA RICORSIONE

LA RICORSIONE

Soluzioni ricorsive di problemi

Quando si può usare una funzione ricorsiva per risolvere un problema?

- se **esiste una soluzione conosciuta** per lo stesso problema di «piccole» dimensioni
 - da questa ricaviamo il **caso base** della funzione
- e se esiste un modo di ottenere la soluzione di un problema di dimensione maggiore **a partire dalla soluzione dello stesso problema di dimensione minore**
 - da questa definizione ricaviamo il **caso ricorsivo**, che è formato da 3 parti:
 - **riduzione** del problema in problemi «più piccoli»
 - **chiamata ricorsiva** della funzione per risolvere i casi «più piccoli»
 - elaborazione delle soluzioni «piccole» per ottenere la **soluzione del problema originale**

Esempio

Funzione fattoriale

Definizione iterativa:

«Il fattoriale di un numero intero N è il prodotto dei numeri 1..N»

```
Ovvero: risultato = 1
         for (i=N; i>0; i--);
             risultato *= i
```

Definizione ricorsiva:

Caso base: «Il fattoriale di 1 è 1»

Riduzione del problema:

«Per moltiplicare i numeri da 1 a N posso prima moltiplicare da 1 a N-1...

Costruzione della soluzione finale a partire da quella ridotta:

... e poi moltiplicare per N»

ovvero

fattoriale(N) = 1	se $N < 2$	(caso base)
fattoriale(N) = $N * \text{fattoriale}(N-1)$	altrimenti	(caso ricorsivo)

Chiamate annidate

Implementazione iterativa

```
# factorial N!

.data

N: .word 10
rez: .word 0

.text

lw a0, N
jal factorial # these two call F(N)

# Save the result
la a6, rez
sw a0, (a6)

#print the result
li a7, 1
ecall

li a7, 10      # se dimentichiamo questo andiamo in un ciclo infinito
ecall
```

Esempio RARS

Implementazione iterativa (continua)

```
factorial:
li a7, 1      # neutro per la moltiplicazione

While:
    beqz a0, EndWhile
    mul a7, a7, a0
    addi a0, a0, -1
    j While
EndWhile:
mv a0, a7
ret          # sarebbe jr ra
```

Implementazione ricorsiva

```
factorial: # factorial(a0: int) : (a1: int) return the factorial of a0
    beqz a0, BaseCase

RecursiveStep: # a0 * factorial(a0-1)
    addi sp, sp, -8
    sw ra, 0(sp)
    sw a0, 4(sp) # qui ci serve salvare a0 perché non siamo foglia

    addi a0, a0, -1
    jal factorial # from here all the recursive calls are done and
                  # the stack is filled, then from the last call each
                  # multiplication is done recovering the value from
                  # the stack. After execution, the result is in a0,
                  # the original a0 is in the stack.

    mv a1, a0
    lw a0, 4(sp)
    lw ra, 0(sp)
    addi sp, sp, 8

    mul a0, a1, a0

    jr ra

BaseCase:
    li a0, 1 # 0! is 1 by definition
    ret
```

Implementazione ricorsiva

```
factorial: # factorial(a0: int) : (a1: int) return the factorial of a0
    beqz a0, BaseCase
```

Salva
dati
su stack

```
RecursiveStep: # a0 * factorial(a0-1)
    {
        addi sp, sp, -8
        sw ra, 0(sp)
        sw a0, 4(sp) # qui ci serve salvare a0 perché non siamo foglia
```

```
    addi a0, a0, -1
    jal factorial # from here all the recursive calls are done and
                  # the stack is filled, then from the last call each
                  # multiplication is done recovering the value from
                  # the stack. After execution, the result is in a0,
                  # the original a0 is in the stack.
```

Recupera
dati
da stack

```
    {
        mv a1, a0
        lw a0, 4(sp)
        lw ra, 0(sp)
        addi sp, sp, 8
```

```
    mul a0, a1, a0
```

```
    jr ra
```

```
BaseCase:
```

```
    li a0, 1 # 0! is 1 by definition
    ret
```

Implementazione ricorsiva

```
factorial: # factorial(a0: ...)
    beqz a0, BaseCase
```

Salva
dati
su stack

```
RecursiveStep: # a0 * fac
    {
        addi sp, sp, -8
        sw ra, 0(sp)
        sw a0, 4(sp) # qui ci
```

```
        addi a0, a0, -1
        jal factorial # from
                        # the
                        # multi
                        # the s
                        # the c
```

Recupera
dati
da stack

```
        {
            mv a1, a0
            lw a0, 4(sp)
            lw ra, 0(sp)
            addi sp, sp, 8
```

```
        mul a0, a1, a0
```

```
        jr ra
```

```
BaseCase:
```

```
    li a0, 1 # 0! is 1 by def
    ret
```

10
ra
9
ra
8
ra
7
ra
...
...
1
ra
X
X
X

Struttura
dello stack

Da ricorsione a iterazione

È sempre possibile trasformare un problema ricorsivo in uno iterativo (e viceversa)

Caso semplice, se la funzione ricorsiva ha una sola chiamata ricorsiva:

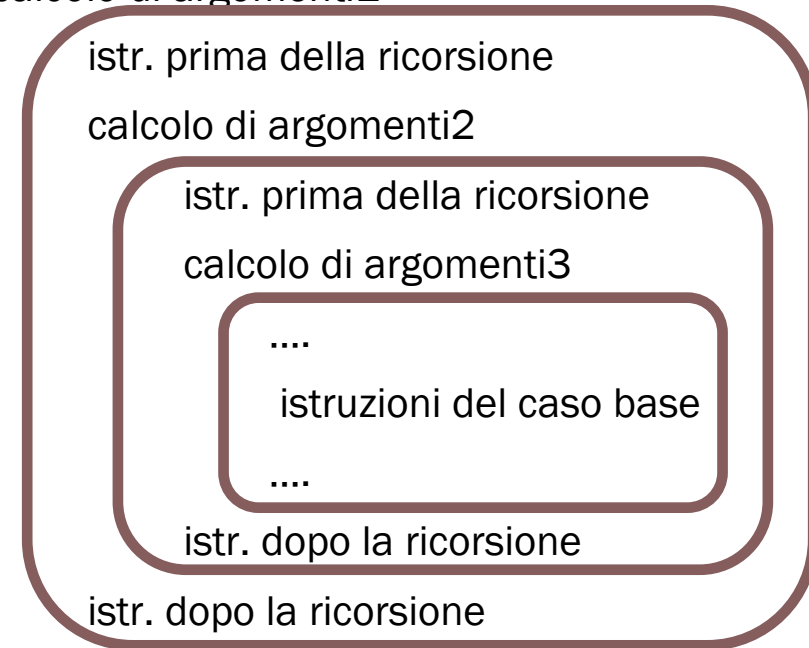
```
funzione( argomenti0 )  
    if caso_base  
        istruzioni del caso base  
        return risultato  
    else  
        istr. prima della ricorsione  
        calcolo di argomenti1  
        funzione( argomenti1 )  
        istr. dopo la ricorsione  
        return risultato
```

L'esecuzione si svolgerà più o meno così:

funzione(argomenti0)

istr. prima della ricorsione

calcolo di argomenti1



istr. dopo la ricorsione

istr. dopo la ricorsione

... versione iterativa

Due cicli eseguono nell'ordine:

- la ripetizione delle **istruzioni precedenti** alla chiamata ricorsiva (compresa di **aggiornamento degli argomenti** della chiamata),
- il **caso base**,
- e poi la ripetizione (per lo stesso numero di volte) delle **istruzioni seguenti** la chiamata ricorsiva (se necessario riottenendo il valore che avevano gli **argomenti**)

funzione(argomenti)

K=0

while (not caso base)

K += 1

istr. prima della ricorsione

aggiornamento degli argomenti

istruzioni del caso base

for (i=0 ; i<K ; i++)

ricostruzione degli argomenti

istr. dopo la ricorsione

NOTA se la ricorsione è multipla, un contatore non è sufficiente per ricostruire la struttura delle chiamate. Può essere necessario usare uno stack per simulare la gestione corretta delle chiamate

Esempio

def fattoriale(N):

if N < 2:

return 1 # caso base

else:

N1 = N - 1 # aggiorno args

F1 = fattoriale(N1) # ricorsione

F2 = F1 * N # istr. dopo ric.

return F2

NOTA: possiamo semplificare la versione iterativa usando la commutatività dell'operazione prodotto per svolgere i prodotti da N a 1 invece che da 1 a N ed eliminando il conteggio non più necessario

def fattoriale(N):

i=0

contatore

while (N >1):

i = i + 1

conto chiamate

N = N - 1

aggiorno args

F1 = 1

caso base

for j in range(i):

N = N + 1

ricostruisco args

F1 = F1 * N

istr. dopo ric.

return F1

def fattoriale(N):

F1 = 1

caso base

while (N >1):

N = N - 1

aggiorno args

F1 = F1 * N

istr. dopo ric.

return F1

Compito per casa



Realizzare in assembly, **sia** in forma **ricorsiva** che in forma **iterativa**, la funzione **GCD** (massimo comun divisore di due interi positivi) definita come segue:

$$\text{GCD}(x, y) = x \quad \text{se } x = y$$

$$\text{GCD}(x, y) = \text{GCD}(y, x) \quad \text{se } x > y$$

$$\text{GCD}(x, y) = \text{GCD}(x, y - x) \quad \text{se } x < y$$

Esempio di esecuzione

$$\text{GCD}(120, 105) =$$

$$\text{GCD}(105, 120) =$$

$$\text{GCD}(105, 15) =$$

$$\text{GCD}(15, 105) =$$

$$\text{GCD}(15, 90) =$$

$$\text{GCD}(15, 75) =$$

$$\text{GCD}(15, 60) =$$

$$\text{GCD}(15, 45) =$$

$$\text{GCD}(15, 30) =$$

$$\text{GCD}(15, 15) = 15$$

Raggruppalettere (basato sull'esame del 21/7/2014)

- Si realizzi la funzione **raggruppalettere** che raggruppa le lettere di una stringa mettendo:
 - prima le lettere (ovvero i caratteri compresi tra 'a' e 'z' e tra 'A' e 'Z')
 - poi i restanti caratteri (tutto il testo comprese lettere accentate, cifre, interpunzione o altro)

Raggruppalettere

Prima

“A B C, easy as 1, 2, 3 / Are simple as do re mi!”

Dopo



Raggruppalettere

Prima

“A B C, easy as 1, 2, 3 / Are simple as do re mi!”

Dopo

“AiBmCereasyoasdsaelpmisera / 3 , 2 , 1 , ! ”



Raggruppalettere (basato sull'esame del 21/7/2014)

- Si realizzi la funzione **raggruppalettere** che raggruppa le lettere di una stringa mettendo:
 - prima le lettere (ovvero i caratteri compresi tra 'a' e 'z' e tra 'A' e 'Z')
 - poi i restanti caratteri (tutto il testo comprese lettere accentate, cifre, interpunzione o altro)
- **NOTA:** l'ordine dei caratteri in ciascuno dei due gruppi non è importante
- **Argomenti da passare (o altro se preferite):**
 - indirizzo della stringa
- **Risultato da restituire:**
 - nessuno (la stringa va modificata in memoria)
- **Esempio di algoritmo ricorsivo:**
 - se la stringa è lunga 1 o 0 caratteri torna senza fare alcunché (è già ordinata)
 - se il primo carattere è una lettera esegui **raggruppalettere** sul resto della stringa (la prima lettera è già al suo posto)
 - se il primo carattere non è una lettera scambiala con l'ultima ed esegui **raggruppalettere** sul resto della stringa più corta di un carattere (l'ultima è stata messa a posto, e del primo non sappiamo alcunché)
- **Esempio di algoritmo iterativo:**
 - Scandisco i caratteri e sposto in fondo quelli che non sono lettere
 - (servono due indici a inizio e fine)

Esempio

A,B,C123 → ,B,C123

A,B,C123 → scambio e ,B,C12

A3B,C12, ...

A2B,C13,

A1B,C23,

ACB,C23,

Codici ASCII (per decidere se il carattere è una lettera)

USASCII code chart

b₇ b₆ b₅ Bits Column Row					0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
b ₄	b ₃	b ₂	b ₁		0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

Codici ASCII (per decidere se il carattere è una lettera)

USASCII code chart

b7b6b5 Bits					000		001		010		011		100		101		110		111	
b4	b3	b2	b1	Column Row	0	1	2	3	4	5	6	7								
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p								
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q								
0	0	1	0	2	STX	DC2	"	2	B	R	b	r								
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s								
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t								
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u								
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v								
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w								
1	0	0	0	8	BS	CAN	(	8	H	X	h	x								
1	0	0	1	9	HT	EM	)	9	I	Y	i	y								
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z								
1	0	1	1	11	VT	ESC	+	;	K	[	k	{								
1	1	0	0	12	FF	FS	,	<	L	\	l									
1	1	0	1	13	CR	GS	-	=	M	]	m	}								
1	1	1	0	14	SO	RS	.	>	N	^	n	~								
1	1	1	1	15	SI	US	/	?	O	_	o	DEL								

Preparazione per la prossima lezione

1. Portate un computer con Rars installato
2. Scaricate la cartella compressa che troverete su classroom
3. Se volete testare il setup che ci sarà all'esame, oltre al punto 1 serve anche uno dei seguenti
 - Su Debian: make, default-jre
 - Su MacOS: XCode o CommandLineTools, openjdk (from brew)
 - Su Windows:
 1. WSL2 con Debian/Ubuntu (make, default-jre)
 2. In alternativa (senza WSL), make e sh sono sufficienti, one-click installer al link seguente: <https://github.com/AlessandroChecco/winmakesh/tree/main>

Java to assembly

```
class HelloWorld {  
    // Class-level (static) variable, accessible in all static methods  
    static int classVariable = 1;  
  
    public static void main(String[] args) {  
        // Method-level (local) variable, only accessible inside main()  
        int methodVariable = 2;  
        methodVariable = classVariable;  
  
        // Calling another method  
        hello();  
    }  
  
    static void hello() {  
        // Function-level (local) variable, only accessible inside hello()  
        int functionVariable = 3;  
  
        functionVariable = classVariable;  
    }  
}
```

Java to assembly

```
HelloWorld::main():
    addi    sp,sp,-32
    sw      ra,28(sp)
    sw      s0,24(sp)
    addi    s0,sp,32
    li      a5,2                # tmp134,
    sw      a5,-20(s0) # tmp134, methodVariable
    lui     a5,%hi(HelloWorld::classVariable) # tmp135,
    lw      a5,%lo(HelloWorld::classVariable)(a5) # tmp136, classVariable
    sw      a5,-20(s0) # tmp136, methodVariable
    call    HelloWorld::hello() #
    nop
    lw      ra,28(sp)
    lw      s0,24(sp)
    addi    sp,sp,32
    jr      ra
HelloWorld::hello():
    addi    sp,sp,-32
    sw      ra,28(sp)
    sw      s0,24(sp)
    addi    s0,sp,32
    li      a5,3                # tmp134,
    sw      a5,-20(s0) # tmp134, functionVariable
    lui     a5,%hi(HelloWorld::classVariable) # tmp135,
    lw      a5,%lo(HelloWorld::classVariable)(a5) # tmp136, classVariable
    sw      a5,-20(s0) # tmp136, functionVariable
    nop
    lw      ra,28(sp)
    lw      s0,24(sp)
    addi    sp,sp,32
    jr      ra
HelloWorld::classVariable:
    .word   1
main:
    addi    sp,sp,-16
    sw      ra,12(sp)
    sw      s0,8(sp)
    addi    s0,sp,16
    call    HelloWorld::main()
    li      a5,0
    mv      a0,a5              #, <retval>
    lw      ra,12(sp)
    lw      s0,8(sp)
    addi    sp,sp,16
    jr      ra
```